

Configuration Raspberry PI 3

Information raspberry :

login : groupe5

mdp : pfr

Utiliser Slam pour cartographier la carte, cette technologie va permettre de concatener la cartographie de la carte pour la construire au fur et à mesure et ne pas reconstruire une nouvelle carte à chaque fois en se déplaçant, perdant l'ancienne.

Connexion : Possibilité d'accéder à la raspberry Pi par terminal de commande (connexion ssh) ou par VNCviewer (environnement graphique)

SSH : ouvrir un terminal de commande, puis se connecter en ssh avec la commande : `ssh groupe5@ipv4`

Moteurs :

Brancher le shield sur la carte arduino (MEGA 2560)

Brancher les câbles des moteurs (rouge et jaune) sur les M1,M2,M3,M4 du shield et le VCC et GND du shield avec les câbles de l'interrupteur de la plateforme.

Caméra :

Activer le service I2c :

-> `sudo raspi-config`

Advanced option -> I2c -> activer

Modifier le fichier config.txt :

-> `sudo nano /boot/firmware/config.txt`

Automatically load overlays for detected cameras

camera_auto_detect=0

dtoverlay=imx219

Redémarrer le raspberry

Vérifier que la caméra est reconnue :

-> `v4l2-ctl --list-devices`

-> `libcamera-hello --list-cameras`

Prendre une photo :

-> `rpicas-still -n -o test.jpg`

Installer les librairies python :

-> `sudo apt install python3-full -y`

Bluetooth :

Faire les branchements entre le composant bluetooth AC-BT-V4 et la carte Arduino.

-> Vcc sur servo_2 "-"

-> GND sur servo_2 "+"
-> Tx sur Rx
-> Rx sur Tx

Code Arduino :

```
#include <AFMotor.h>

// Capteur 1
#define TRIG1 23
#define ECHO1 24

// Capteur 2
#define TRIG2 48
#define ECHO2 50

AF_DCMotor m1(1);
AF_DCMotor m2(2);
AF_DCMotor m3(3);
AF_DCMotor m4(4);

// MOTEURS

void avancer() {
  m1.run(FORWARD);
  m2.run(FORWARD);
  m3.run(FORWARD);
  m4.run(FORWARD);
}

void reculer() {
  m1.run(BACKWARD);
  m2.run(BACKWARD);
  m3.run(BACKWARD);
  m4.run(BACKWARD);
}

void gauche() {
  m1.run(FORWARD);
  m2.run(FORWARD);
  m3.run(FORWARD);
  m4.run(BACKWARD);
}

void droite() {
  m1.run(FORWARD);
  m2.run(BACKWARD);
  m3.run(BACKWARD);
  m4.run(FORWARD);
}

void stopMoteurs() {
  m1.run(RELEASE);
  m2.run(RELEASE);
  m3.run(RELEASE);
  m4.run(RELEASE);
}

// SETUP
void setup() {
  Serial.begin(115200); // Bluetooth avec le 115200
  Serial.begin(115200); // Connection MP10

  m1.setSpeed(200);
  m2.setSpeed(200);
  m3.setSpeed(200);
  m4.setSpeed(200);

  stopMoteurs();

  pinMode(TRIG1, OUTPUT);
  pinMode(ECHO1, INPUT);

  pinMode(TRIG2, OUTPUT);
  pinMode(ECHO2, INPUT);
}

// LOOP BLUETOOTH
void loop() {
  float opt = measureDistance(TRIG1, ECHO1);
  float opt2 = measureDistance(TRIG2, ECHO2);

  Serial.print("Capteur 1 : ");
  Serial.println(opt);

  Serial.print("Capteur 2 : ");
  Serial.println(opt2);

  Serial.println("-----");

  delay(100);

  if (Serial.available()) {
    int c = Serial.read();

    Serial.print("Commande recue : ");
    Serial.println(c);

    switch (c) {
      case 'F': // Forward
        if (opt1 == 10.0) {
          avancer();
        } else {
          Serial.println("Obstacle devant ?");
        }
        break;

      case 'R': // Backward
        if (opt1 == 10.0) {
          reculer();
        } else {
          Serial.println("Obstacle derriere ?");
        }
        break;

      case 'L': // Left
        gauche();
        break;

      case 'D': // Right
        droite();
        break;

      case 'S': // Stop
        stopMoteurs();
        break;

      default:
        stopMoteurs();
        break;
    }
  }
}
```

Utiliser une application de BLE (par exemple Serial Bluetooth Terminal sur téléphone) et envoyer les informations nécessaires pour faire se déplacer le robot. (Faire attention au format d'envoi de la data).

SLAM :

1 - Installation de docker :

```
-> sudo apt update  
-> sudo apt install -y docker.io docker-compose  
-> sudo usermod -aG docker $USER
```

Se déconnecter/reconnecter

Puis vérifier :

```
-> groups # docker doit apparaître  
-> docker run hello-world
```

2 - Télécharger l'image de ROS2

```
-> docker pull arm64v8/ros:humble #Version arm64 obligatoire car on est sur un OS debian en 64bit
```

3 - Créer le docker (environnement dans lequel on va installer et utiliser le SLAM, obligatoire pour utiliser ROS2 qui ne fonctionne que fonctionne pas sur l'os Debian, on passe donc par un docker sur Ubuntu)

```
-> mkdir -p ~/ros2_ws  
  
-> docker run -it \  
    --name ros2_lidar \  
    --network host \  
    --privileged \  
    -v /dev:/dev \  
    -v ~/ros2_ws:/ros2_ws \  
    arm64v8/ros:humble \  
    bash
```

4 - Installer les librairies dans le docker

```
-> source /opt/ros/humble/setup.bash  
-> apt update && apt install -y \  
    git \  
    python3-colcon-common-extensions \  
    python3-rosdep \  
    ros-humble-slam-toolbox \  
    ros-humble-nav2-map-server
```

Ajouter au bashrc pour ne pas refaire à chaque session

```
-> echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc  
-> echo "source /ros2_ws/install/setup.bash" >> ~/.bashrc
```

5 - Cloner et compiler RPLidar

```
-> mkdir -p /ros2_ws/src  
-> cd /ros2_ws/src  
-> git clone https://github.com/Slamtec/sllidar_ros2.git  
-> cd /ros2_ws  
-> colcon build --symlink-install --parallel-workers 1  
  
-> source /ros2_ws/install/setup.bash
```

6 - Configurer SLAM

```
-> mkdir -p /ros2_ws/config  
-> cat > /ros2_ws/config/slam_params.yaml << 'EOF'  
slam_toolbox:  
  ros__parameters:  
    mode: mapping
```

```

odom_frame: odom
map_frame: map
base_frame: base_link
scan_topic: /scan
use_scan_matching: true
use_scan_barycenter: true
resolution: 0.05
max_laser_range: 6.0
minimum_travel_distance: 0.0
minimum_travel_heading: 0.0
map_update_interval: 1.0
use_multithread: false
publish_period_sec: 1.0
EOF

```

7 - Launch file

```

-> mkdir -p /ros2_ws/launch
-> cat > /ros2_ws/launch/slam_launch.py << 'EOF'
from launch import LaunchDescription
from launch_ros.actions import Node
from launch.actions import IncludeLaunchDescription
from launch.launch_description_sources import PythonLaunchDescriptionSource
import os
from ament_index_python.packages import get_package_share_directory

def generate_launch_description():

    lidar = IncludeLaunchDescription(
        PythonLaunchDescriptionSource([
            os.path.join(get_package_share_directory('slidar_ros2'),
                'launch', 'slidar_a1_launch.py')
        ])
    )

    static_tf_laser = Node(
        package='tf2_ros',
        executable='static_transform_publisher',
        name='static_tf_laser',
        arguments=['0', '0', '0', '0', '0', '0', 'base_link', 'laser']
    )

    static_tf_odom = Node(
        package='tf2_ros',
        executable='static_transform_publisher',
        name='static_tf_odom',
        arguments=['0', '0', '0', '0', '0', '0', 'odom', 'base_link']
    )

    slam = Node(
        package='slam_toolbox',
        executable='async_slam_toolbox_node',
        name='slam_toolbox',
        parameters=[
            '/ros2_ws/config/slam_params.yaml',
            {'use_sim_time': False}
        ],
        output='screen'
    )

    return LaunchDescription([static_tf_odom, static_tf_laser, lidar, slam])
EOF

```

8 - Utiliser le lidar

```

-> docker start ros2_lidar
-> docker exec -it ros2_lidar bash
-> chmod 666 /dev/ttyUSB0
-> source /opt/ros/humble/setup.bash
-> source /ros2_ws/install/setup.bash
-> ros2 launch /ros2_ws/launch/slam_launch.py

```

Scanner la pièce avec le robot

Affichage de la carte :

```

-> docker exec -it ros2_lidar bash
-> source /opt/ros/humble/setup.bash
-> source /ros2_ws/install/setup.bash
-> ros2 run nav2_map_server map_saver_cli -f /ros2_ws/ma_carte

```